ML Algorithm used for the model- Support Vector Machine (SVM) – LinearSVC

1. Algorithm Type & Objective

-Type: Supervised Machine Learning algorithm for classification and regression tasks.

-Objective: Find the optimal decision boundary (hyperplane) that maximizes the margin between different classes in the feature space.

In this project, it is used for text classification to predict review ratings.

2.How the Algorithm Works (Mathematical Intuition)

-SVM maps input data into a high-dimensional feature space and tries to find a hyperplane that separates the classes with the largest margin.

-Margin = Distance between the hyperplane and the nearest points from any class (called support vectors).

-For a binary classifier, the decision function is:

$$f(x) = w.x + b$$

Where:

w→ weight vector

b→ bias

Decision rule:

$$Predict\ class\ 1\ if\ \ f(x) \geq 0, else\ class - 1$$

In text classification, TF-IDF features are used as inputs to the SVM.

3. Key Hyperparameters

-C → Regularization parameter.

Small C → larger margin, may misclassify some points.

Large C → smaller margin, tries to classify all points correctly but may overfit.

-loss → Loss function type.

"hinge" → standard SVM loss.

"squared_hinge" → penalizes misclassifications more strongly.

-max_iter → Maximum number of iterations for optimization.

-tol → Tolerance for stopping criterion.

4. Strengths & Limitations

Strengths:

-Works well for high-dimensional data like text.

-Effective even when the number of features >> number of samples.

-Robust against overfitting if C is tuned properly.

Limitations:

-Training can be slow for very large datasets.

-Not naturally designed for multi-class classification (handled internally with one-vs-rest or one-vs-one).

-Performance can degrade if classes are highly imbalanced unless class weights are adjusted.

5. When to Use / When Not to Use

Use When:

-Data is high-dimensional (e.g., text features from TF-IDF).

-The number of samples is moderate.

-You want a robust linear decision boundary.

Avoid When:

-Dataset is extremely large (training can be slow).

-Classes are heavily imbalanced without proper reweighting.

-Decision boundaries are highly non-linear (kernel SVM would be better).